


# Dossier de projet & mode opératoire

Jeu multijoueur · Boutique de skins · Amis & discussion

Conception, règles, parcours joueur, économie, architecture et maintenance. Une référence pratique pour utiliser le site et poursuivre son développement.







**Édition du 24 octobre 2026 · Emmanuel Galland**

Version documentaire correspondant aux fonctionnalités du code, y compris le panneau de discussion et l’ajout d’ami en salon.

https://kikalmo.emmanuel-galland117.chatgpt.site

https://github.com/egalland/kikalmo

# Un jeu de mots, un univers à soi.

Kikalmo transforme des mots privés de leurs voyelles en parties courtes, partagées et personnalisables. Le personnage en laine donne une identité au joueur ; les salons, les amis et la collection soutiennent le retour au jeu.

## Objectif du projet

Proposer un jeu accessible sans création de compte classique : choisir un pseudo, personnaliser son avatar, rejoindre un salon et retrouver les réponses avant les autres. Le mot du jour offre un défi indépendant, sans chronomètre.

| Public / contexte | Réponse du produit |
|---|---|
| Découverte en solo | Une partie démarre même sans autre joueur. |
| Partie entre proches | Codes amis, demandes réciproques et rejoindre un ami. |
| Petites sessions régulières | 10 manches, progression en pièces et collection de skins. |
| PC et mobile | Menu commun, clavier mobile et discussion réductible. |

## Périmètre disponible

13 salons : le généraliste regroupe 6 000 réponses ; les 12 salons thématiques proposent chacun 500 entrées. La boutique contient 49 articles, dont 11 articles Halloween. Le mot du jour dispose d’une banque distincte de 48 expressions et proverbes.

Le profil, les achats et les amis sont liés au navigateur. La clé de statistiques personnelles ne transfère pas le personnage, le portefeuille ou les amitiés.

## État de cette édition

Ce dossier décrit les fonctionnalités présentes dans le code au 24 octobre 2026, y compris la discussion en bas d’écran et l’ajout d’ami depuis un salon. Les essais d’animation textile restent des démonstrations séparées ; ils ne sont pas intégrés au site.

# Le fonctionnement d’une partie.

Le serveur décide de la manche, du temps restant, des bonnes réponses, des points et des récompenses. Le navigateur affiche la partie et transmet les actions du joueur.

| Étape | Comportement attendu |
| --- | --- |
| Entrée dans le salon | 30 joueurs maximum ; arrivée en cours avec 0 point. |
| Compte à rebours | 10 secondes au démarrage d’une nouvelle partie. |
| Recherche | 10 manches de 20 secondes maximum chacune. |
| Indice | Un indice apparaît à 10 secondes restantes. |
| Révélation | 5 secondes ; immédiate si tous les joueurs ont trouvé. |
| Podium | Classement affiché pendant 60 secondes avant la suite. |

## Validation des réponses

La proposition doit retrouver le mot ou l’expression avec ses voyelles. La normalisation ignore la casse, les accents, les espaces, la ponctuation et les ligatures œ/æ. Une faute est tolérée à partir de 8 lettres. Une proposition voisine peut produire un retour « Proche ». Une autre réponse valide du même thème n’est pas acceptée comme une simple faute.

## Points et classement

Une bonne réponse rapporte de 1 000 à 100 points selon la rapidité. Les points de toutes les manches s’additionnent. Le premier à résoudre la manche est identifié distinctement. La couronne du vainqueur précédent reste visible tant que ce joueur est dans le salon.

## Présence et reprise

Après 12 secondes sans présence actualisée, un joueur quitte la liste active. Si le salon devient vide, la partie reste en pause pendant 10 secondes. Revenir pendant cette fenêtre permet de reprendre la partie ; après son expiration, le salon repart de zéro.

Les points du classement, les pièces de boutique et les étoiles du mot du jour sont trois valeurs différentes. Dépenser des pièces ne modifie jamais le score d’une partie.

# Personnage, collection et navigation.

L’identité textile traverse le logo, les personnages, les monnaies, les contrôles et les surfaces. La boutique reste une interface d’achat lisible, sans slogans de mercerie.

## Personnalisation du personnage

Le joueur compose son visage, ses cheveux, ses yeux, sa bouche, son nez, ses vêtements, ses accessoires et sa couleur. Des variantes de collection ajoutent coiffures et regards. Les éléments verrouillés ouvrent la boutique ; un achat ne les équipe pas automatiquement.

| Emplacement de collection | Règle de composition |
|---|---|
| Chapeau ou casque | Conserve les cheveux compatibles et la forme du visage. |
| Broche ou effet | Se place au-dessus des vêtements et costumes. |
| Costume | Habille le personnage sans remplacer son identité. |
| Coiffure / regard premium | Remplace uniquement la partie correspondante. |
| Motif textile | Suit la surface du personnage et son contour. |

## Navigation commune

Le menu burger est proposé sur PC et mobile. Il donne accès aux salons, au personnage, à la boutique et aux statistiques. Les salons sont triés selon leur fréquentation ; celui qui rassemble le plus de monde est mis en avant. Les amis présents apparaissent avec leur avatar sur les cartes des salons.

## Direction artistique

Fond ivoire, feutre texturé, coutures discrètes, vert sauge et détails dorés. Les boutons restent contrastés et les textes fonctionnels. Les images d’avatar sont composées à partir de véritables assets transparents. Les zones de visage restent ajustées à la forme choisie, y compris dans un costume.

Le logo Halloween reprend le mot-symbole Kikalmo en laine, avec un chapeau et une citrouille. Il est activé pendant la saison 2026 et revient au logo habituel à la fin de l’événement.

# Démarrer et jouer.

Parcours de première utilisation, depuis l’accueil jusqu’au classement de la partie.

1. Ouvrir Kikalmo, puis créer son personnage si aucun profil n’est enregistré sur ce navigateur.
2. Saisir un pseudo de 1 à 20 caractères. Choisir les éléments du personnage, puis enregistrer.
3. Choisir un salon depuis les cartes de l’accueil ou le menu. Le salon le plus fréquenté est mis en avant.
4. Attendre la manche, lire les consonnes et proposer le mot complet dans le champ de réponse.
5. Lire le retour : bonne réponse, proposition proche ou nouvelle tentative. Les essais sont gratuits.
6. Utiliser l’indice après 10 secondes si nécessaire. La révélation montre la réponse et peut présenter une illustration.
7. Rester jusqu’au classement pour terminer la partie et recevoir le bonus de participation si les conditions sont remplies.

## Raccourcis et confort

Entrée envoie la proposition. La flèche haut rappelle le dernier message lorsque le champ est vide. Sur mobile, le clavier du site propose une saisie adaptée et une option pour utiliser le clavier natif. Les sons peuvent être désactivés ; les réactions apparaissent sur les personnages.

## Changer de salon ou de personnage

Le menu permet de rejoindre un autre salon ; la nouvelle session devient active et l’ancienne est quittée. Modifier le personnage conserve le score de la partie en cours. L’enregistrement met à jour le pseudo et l’avatar visibles par les autres joueurs.

En cas de connexion interrompue, attendre le retour du service puis rejoindre le salon. Une reprise complète dépend de la fenêtre de présence et du délai de pause de la partie.

# Discuter et retrouver ses amis.

La discussion réductible se trouve en bas de l’écran dans un salon. Le bouton d’ajout d’ami est disponible à côté des autres joueurs dans le classement.

## Utiliser la fenêtre de discussion

1. Cliquer sur Discussion en bas de l’écran pour ouvrir la fenêtre du salon.
2. Lire les messages ; les propositions de réponse restent dans le fil principal et ne sont pas recopiées dans cette fenêtre.
3. Pendant la recherche, trouver le mot pour pouvoir écrire. Les échanges sont alors visibles seulement aux joueurs qui ont trouvé.
4. À la révélation, les échanges deviennent visibles à tous. Écrire un message de 200 caractères maximum, puis envoyer.
5. Réduire la fenêtre pour retrouver toute la zone de jeu. Le compteur indique les nouvelles arrivées de messages lorsque la fenêtre est réduite.

## Ajouter un joueur du salon

1. Repérer le joueur dans le classement, puis cliquer sur le bouton Ajouter en ami à côté de son nom.
2. La demande est envoyée à ce joueur ; une icône d’attente remplace le bouton d’ajout.
3. Le destinataire ouvre Mes amis et accepte ou refuse. L’expéditeur ne peut pas accepter à sa place.
4. Une fois la demande acceptée, retrouver cet ami dans le panneau Amis et utiliser Rejoindre sa partie.

## Codes amis et contrôles

Le panneau Amis permet aussi de copier son code, d’ajouter un code reçu, d’annuler une demande, de retirer un ami ou de le bloquer. La présence peut être masquée. Le serveur vérifie le salon, la présence récente, les blocages et la capacité avant de permettre de rejoindre un ami.

La fenêtre est une discussion de salon, pas une messagerie privée entre deux amis. Un ajout depuis un salon prépare un contact avec le pseudo et l’avatar publics, sans transformer la demande en amitié automatique.

# Acheter, puis équiper.

La boutique sert à acheter des skins. Mon personnage sert à les porter. Les deux écrans restent accessibles depuis la navigation.

1. Ouvrir Boutique. Tous les articles disponibles sont affichés par défaut.
2. Choisir éventuellement une catégorie : Halloween, costumes, chapeaux, broches, coiffures, regards ou motifs.
3. Cocher les monnaies souhaitées. Les deux cases peuvent rester cochées pour voir simultanément les achats en pièces et en étoiles.
4. Cliquer sur un article pour l’examiner. Les flèches de l’aperçu parcourent les articles qui correspondent aux filtres.
5. Vérifier le prix et le solde correspondant. Cliquer sur Acheter lorsque le solde est suffisant.
6. Ouvrir Mon personnage. Sélectionner l’article débloqué dans son emplacement, puis enregistrer.

## Collection Halloween 2026

La collection contient 11 articles. Les achats sont disponibles jusqu’au 9 novembre inclus, heure de Paris, et s’arrêtent le 10 novembre 2026 à 00 h. Le serveur applique cette limite, même si un ancien écran de boutique est resté ouvert. Les articles déjà achetés restent possédés et équipables après la fin de la vente.

## Comportements attendus

Un aperçu ne change pas le personnage. Un achat réussi ne débite qu’une fois, y compris après un nouvel envoi de la même demande. Un article déjà possédé n’est pas racheté. Les achats vérifient le tarif et le solde côté serveur ; modifier le prix dans le navigateur ne permet pas de contourner la boutique.

Les pièces et les étoiles ne sont pas interchangeables. La possession est liée à l’identifiant de ce navigateur ; vider les cookies peut faire perdre l’accès à la collection.

# Une progression accessible, des objectifs durables.

Équilibrage des pièces au 24 octobre 2026. Les récompenses et les prix en étoiles restent inchangés.

| Condition | Gain en pièces |
|---|---|
| Terminer avec une proposition dans 5 manches distinctes | +3 en fin de partie |
| À plusieurs : trouver le mot en premier | +1 par manche |
| En solo : trouver 3 mots distincts | +1 aux seuils 3, 6 et 9 |
| Maximum à plusieurs / en solo | 13 / 6 par partie |

## Conditions et anti-doublons

Les propositions vides, les réactions et les simples présences ne comptent pas comme participation. Le bonus de fin exige de rester jusqu’au podium. Une reconnexion ne permet pas de recompter une manche ni de recevoir deux fois un bonus. Revenir après la fin ne donne pas rétroactivement le bonus de présence.

| Famille | Prix en pièces | Repère à 5 pièces / partie |
|---|---|---|
| Accessoires | 8 à 40 | 2 à 8 parties |
| Coiffures et regards | 45 à 75 | 9 à 15 parties |
| Costumes Halloween | 60 à 80 | 12 à 16 parties |
| Costumes permanents | 180 à 550 | 36 à 110 parties |

## Exemples

Un joueur présent, avec au moins cinq manches tentées et aucune première place, gagne 3 pièces. Trois premières places dans une partie à plusieurs portent le total à 6 pièces. Un joueur solo qui trouve dix mots et termine la partie gagne 6 pièces. Les durées d’acquisition dépendent de la réussite ; le repère à cinq pièces est une hypothèse de comparaison.

La hausse des prix ne retire aucun achat et ne débite aucun solde existant. Le remboursement historique de la première baisse Halloween est conservé ; les nouveaux achats en pièces n’y sont pas éligibles.

# Le mot du jour et les statistiques.

Le défi quotidien est indépendant des salons et comporte une difficulté plus élevée, centrée sur les expressions et les proverbes.

## Résoudre le défi du jour

1. Depuis l’accueil, ouvrir Le mot du jour. Le défi ne comporte pas de chronomètre.
2. Proposer l’expression complète. La progression est conservée pour ce navigateur.
3. Après 3 puis 6 erreurs, lire les indices supplémentaires. Après 9 erreurs, une voyelle est révélée.
4. Consulter le résultat et le classement. Le nombre d’essais prime, puis l’heure de la bonne réponse départage les joueurs.

| Classement quotidien | Étoiles après clôture |
|---|---|
| Premier | 5 |
| Deuxième | 3 |
| Troisième | 2 |
| Autre participant avec au moins une proposition | 1 |

## Clôture et renouvellement

Un nouveau défi commence à minuit, heure de Paris. Les récompenses sont calculées sur la journée clôturée et ne sont attribuées qu’une fois. Aucune récompense provisoire de podium n’est versée en journée. Si personne n’a trouvé, l’encart de tête affiche simplement « La place est libre. ».

## Lire et conserver ses statistiques

Le suivi statistique dépend du consentement facultatif. Mes statistiques permet de consulter son historique et d’utiliser une clé privée pour regrouper cet historique sur plusieurs appareils. La clé ou la sauvegarde JSON doit être conservée par le joueur. Elle ne fusionne pas les profils, les pièces, les achats ou les amis.

Retirer le consentement aux statistiques ne désactive pas le jeu ni la progression fonctionnelle. Les règles d’export et d’effacement restent accessibles depuis le pied de page.

# Un serveur de jeu, un état de référence.

Application React et TypeScript avec Vinext, hébergée via Sites dans un Worker Cloudflare. La base D1 conserve l’état des salons et les données fonctionnelles.

| Couche | Rôle / fichiers principaux |
| --- | --- |
| Interface | app/page.tsx ; composants Avatar, DailyWord, CosmeticShop, FriendsPanel, RoomDiscussion. |
| Règles de jeu | lib/game.ts : cycle des manches, validation, scores et progression des pièces. |
| Économie | lib/economy.ts et lib/cosmetics.ts : livre des crédits, achats, catalogue et échéances. |
| Social | lib/friends.ts ; API friends : relations, présence et demandes depuis un salon. |
| Données | db/schema.ts ; migrations Drizzle ; base D1 liée sous DB. |
| Mesure / droits | lib/analytics.ts, lib/privacy.ts et API privacy. |

## Flux d’une action

1. Le navigateur envoie une action à l’API. 2. Le serveur reconnaît le joueur via son cookie et lit la révision du salon. 3. Il avance le temps, applique l’action et calcule les effets. 4. Une mise à jour conditionnelle valide la révision. 5. Les écritures liées, dont les récompenses, sont exécutées dans le même lot D1. 6. Le serveur renvoie une vue filtrée et sa nouvelle révision.

## Concurrence

Le contrôle compare-and-swap évite que deux réponses simultanées écrasent l’état. Une écriture perdante est recalculée à partir de la dernière révision. Les récompenses vérifient aussi une marque de validation propre au lot ; un essai perdant ne peut pas créditer le joueur. Les identifiants de récompense et d’achat empêchent les doublons.

La synchronisation de partie repose sur des requêtes régulières, pas sur un WebSocket. Les réponses API ne mettent jamais le stock de réponses ni les cookies de session à disposition du navigateur.

# API, état et persistance.

Les opérations d’écriture utilisent les contrôles d’origine et l’identité côté serveur. Les identifiants courts visibles dans le classement ne sont pas les cookies des joueurs.

| Route | Opérations principales |
| --- | --- |
| /api/game | Lister ; join, sync, answer, leave, profile, chat, emote. |
| /api/daily | Lire le défi et le classement ; rejoindre et proposer une réponse. |
| /api/economy | Lire les soldes et possessions ; acheter un article. |
| /api/friends | Lire ; register, pulse, request, request-room, accept, decline, cancel, remove, block, unblock, visibility, locate. |
| /api/privacy | Choix statistiques ; export associé au navigateur ; effacement. |
| /api/admin/auth | Authentification et session privée d’administration. |

## Données fonctionnelles

rooms conserve le JSON de partie et sa révision. economy_ledger conserve les crédits et débits par monnaie, avec les identifiants uniques d’achat et de récompense. daily_challenges et daily_players conservent le défi, les essais et les résultats quotidiens. friend_profiles, friend_links et friend_blocks conservent les contacts publics, demandes, amitiés acceptées et blocages.

## État de partie non public

Le serveur conserve le paquet de questions, les identifiants complets et la progression de récompenses. La vue publique masque ces données. Elle ne transmet que la progression en pièces du joueur courant. La réponse n’est dévoilée qu’à la révélation ou au podium ; les discussions entre gagnants sont filtrées côté serveur.

## Statistiques

Les tables analytics_* conservent les mesures pseudonymisées et les agrégats. personal_events, personal_sessions et personal_daily portent l’historique personnel ; personal_stat_keys et player_stat_links permettent son association à une clé. site_visitor_days mesure les visites consenties. admin_login_limits protège l’entrée administration.

# Des fonctions séparées, des droits accessibles.

La progression et le social font fonctionner le service. La mesure d’audience reste facultative. Ce dossier décrit le comportement technique ; les textes publics doivent être complétés par l’éditeur avant un lancement officiel.

## Identité et accès

Le cookie sv_player est HttpOnly, Secure et SameSite=Lax, avec une durée de 30 jours lorsqu’il est renouvelé. Le profil visuel est aussi conservé localement. Aucun mot de passe joueur ni e-mail n’est requis. Le code ami public est différent de l’identifiant de session. Une amitié nécessite l’acceptation du destinataire.

| Données | Durée / comportement |
|---|---|
| Défi quotidien | 30 jours de détails. |
| Salons inactifs | Nettoyage après 24 h d’inactivité, lors d’une visite suivante. |
| Détails de mesure d’audience | 90 jours maximum. |
| Historique personnel / agrégats | 395 jours maximum. |
| Session administration | 12 heures ; fenêtre de limitation de connexion de 15 minutes. |
| Collection / amis | Liés au navigateur ; effacement via les commandes de confidentialité. |

## Mode opératoire : exporter ou effacer

1. Ouvrir Confidentialité et cookies depuis le pied de page.
2. Exporter les données associées au navigateur avant de supprimer ses cookies, si une copie est souhaitée.
3. Retirer le consentement pour supprimer les mesures facultatives de ce navigateur tout en conservant les fonctions de jeu.
4. Utiliser l’effacement complet pour supprimer les éléments fonctionnels associés : profil, participations, collection, relations et messages encore conservés.

Les purges s’exécutent lors des visites et écritures suivantes. Sans trafic, les données expirées attendent une prochaine requête. Le contact public, les mentions d’éditeur et les informations d’hébergement sont encore à compléter dans lib/publication.ts.

# Modifier, vérifier et publier.

Les sources du Site et le dépôt GitHub main doivent rester synchronisés. Les secrets ne doivent jamais être écrits dans les sources ou dans ce dossier.

## Préparer une modification

1. Récupérer la version source courante du Site et vérifier les modifications locales avant de travailler.
2. Identifier le module concerné : interface, jeu, économie, social ou statistiques. Préserver les identifiants des articles et les monnaies des achats existants.
3. Si le schéma D1 change, modifier db/schema.ts puis générer une migration avec npm run db:generate. Conserver le SQL, le snapshot et le journal générés.
4. Appliquer les changements, puis exécuter le contrôle TypeScript et les tests du périmètre. Lancer la compilation de production avec le helper Sites.
5. Pousser l’état source vérifié, créer l’archive de cette révision, enregistrer la version puis la déployer. Vérifier le statut final de publication.
6. Synchroniser les mêmes modifications sur GitHub main, en conservant les références aux révisions sources et les particularités du README GitHub.

## Commandes de contrôle

TypeScript : npx tsc --noEmit. Jeu et API : node --test tests/game.test.mjs tests/api.test.mjs. Économie : node --test tests/economy.test.mjs. Social : node --test tests/friends.test.mjs tests/room-discussion.test.mjs. Confidentialité : node --test tests/privacy.test.mjs. Les vérifications visuelles du navigateur complètent ces contrôles lorsqu’un aperçu est disponible.

## Administration

La page /admin utilise une authentification protégée. Configurer KIKALMO_ADMIN_PASSWORD avec au moins 16 caractères et KIKALMO_ADMIN_SESSION_SECRET dans l’environnement d’hébergement. Ne pas réutiliser les anciennes valeurs de démonstration. Les rapports permettent d’analyser l’activité et d’exporter des résultats selon les fonctions présentes.

Avant une opération de base de données, préparer un export et vérifier la restauration avec les outils de l’hébergeur. La synchronisation des sources ne constitue pas une sauvegarde des données D1.

# Vérifier les parcours, documenter les limites.

Les tests automatiques valident les règles et les écritures. Une recette PC et mobile reste nécessaire pour contrôler la lisibilité, les positions et les interactions visuelles.

| Parcours | Résultat attendu |
| --- | --- |
| Deux bonnes réponses simultanées | Une seule première place et un seul crédit associé. |
| Partie avec 5 manches tentées | +3 au podium ; aucun doublon après actualisation. |
| Solo, dix mots trouvés | 3 crédits de seuil + 3 de participation. |
| Demande d’ami dans un salon | Présence commune récente requise ; destinataire seul habilité à accepter. |
| Blocage / présence masquée | Les préférences restent appliquées après un ajout depuis le salon. |
| Discussion pendant une manche | Saisie verrouillée avant résolution ; messages filtrés selon le statut. |
| Achat concurrent / collection expirée | Pas de solde négatif ni de nouvel achat Halloween hors délai. |
| Effacement des données | Les données du joueur sont retirées sans supprimer celles des autres. |

## Recette visuelle à effectuer

PC : ouvrir et réduire la discussion, lire le classement, envoyer une demande, accepter dans un second navigateur, rejoindre un ami et changer de salon. Mobile : vérifier à 320 px et sur un écran courant, avec clavier natif et clavier du site ; contrôler que les champs, boutons et modales restent accessibles. Ouvrir l’atelier au-dessus de la navigation et contrôler le défilement unique.

## Limites connues et décisions différées

Le profil, la collection et les amitiés ne sont pas transférables entre appareils par un compte. La discussion n’est pas une messagerie privée persistante. Les illustrations externes de révélation peuvent être indisponibles. Le module d’administration dépend de ses variables d’environnement. Les animations de tricotage restent hors du site.

Lors de cette édition, l’aperçu navigateur de l’environnement était indisponible. Les tests serveur, le contrôle TypeScript et la compilation peuvent être vérifiés ; ils ne remplacent pas une validation visuelle sur les appareils réels.

# Structure du projet et exécution.

Repères pour reprendre le développement à partir des sources. Les versions exactes des dépendances sont celles du package.json et du verrou de dépendances du dépôt.

| Emplacement | Responsabilité |
| --- | --- |
| app/ | Pages React, composants client, styles et routes API. |
| lib/ | Moteur de jeu, profils, économie, social, statistiques et règles partagées. |
| db/schema.ts + drizzle/ | Schéma D1 déclaré et historique de migrations versionné. |
| public/puppets/ + public/branding/ | Sprites, costumes, accessoires, logo et textures optimisés. |
| tests/ + scripts/ | Tests Node, contrôles d’avatars, génération et outils de construction. |
| .openai/hosting.json | Identité du Site et déclarations des ressources d’hébergement. |

## Pile et prérequis

Le package déclare Node.js >= 22.13.0. L’interface utilise React, TypeScript et les routes de type App Router de Vinext ; Vite construit les environnements client et serveur. Les routes importent env depuis cloudflare:workers : un simple serveur de fichiers statiques ne suffit donc pas pour faire fonctionner le multijoueur, les achats et les amis.

## Environnements à distinguer

Le développement et les tests peuvent utiliser un stockage local ou une base SQLite simulée. La production utilise le binding D1 DB. Les données de test ne doivent pas être confondues avec les achats et parties de production. Le script npm start démarre Wrangler en mode local à partir du résultat de compilation ; ce n’est pas une procédure de déploiement de production.

L’application actuelle est hébergée sur Sites / Workers avec D1. Un transfert sur VPS impose d’adapter les accès Cloudflare et la base ; copier uniquement les fichiers publics ne recrée pas le service.

# Clés, contraintes et relations D1.

Le schéma de référence est db/schema.ts. Les colonnes state et avatar contiennent du JSON sérialisé ; les autres colonnes structurent les recherches, contraintes et agrégats.

| Table / clé | Colonnes et invariant |
|---|---|
| rooms · id | state, revision ; une ligne par salon, écriture conditionnée par revision. |
| economy_ledger · id | player, currency, delta, item, created_at ; identifiant unique de chaque crédit ou achat. |
| daily_challenges · day | question ; un défi commun pour une date de Paris. |
| daily_players · id | day, player, name, avatar, state, revision, tries, solved_at ; tentatives d'un navigateur pour un jour. |
| friend_profiles · player | code UNIQUE, name, avatar, seen, visible ; le code public ne remplace pas le cookie. |
| friend_links · (requester, recipient) | status, created_at ; une direction canonique par paire, index destinataire/statut. |
| friend_blocks · (owner, target) | Un blocage dans un sens interdit les demandes et la localisation entre les deux joueurs. |

## Relations logiques

Le champ player relie les achats, défis et contacts à la session du navigateur. Ces liens sont gérés par le code applicatif ; ne pas supposer l'existence de cascades de clés étrangères SQL. L'effacement complet supprime explicitement les lignes liées et nettoie les salons. Les tableaux de joueurs et messages restent dans rooms.state plutôt que dans des tables séparées.

## Évolution du schéma

Modifier la déclaration Drizzle, exécuter npm run db:generate, puis relire la migration SQL et les métadonnées générées. Versionner ensemble la migration, le snapshot et le journal. Préparer une sauvegarde des données avant application. Ne pas réécrire une migration déjà appliquée et ne pas créer les tables à la volée dans une route API.

Les sauvegardes de code et les sauvegardes D1 répondent à deux besoins différents. Un retour à une ancienne version du code ne retire pas automatiquement une migration ni une transaction déjà écrite.

# Exemples d’appels et traitement des erreurs.

Exemples de structure, avec valeurs fictives. Les champs game et round doivent provenir du dernier instantané de partie reçu du serveur.

### Proposer une réponse

```
POST /api/game
Content-Type: application/json
{
  "action": "answer", "room": "cinema",
  "game": "identifiant-du-match-recu",
  "round": 3, "answer": "Une proposition"
}
```

La réponse comprend l’état public, revision, serverNow, result, roomCoinsEarned et roomCoinProgress pour le joueur courant. result peut notamment valoir correct, close, incorrect ou wait. Les gains n’obéissent jamais à un montant envoyé par le client.

Achat : POST /api/economy avec {"item":39}. Le portefeuille renvoyé contient room, daily, owned et serverNow. Les valeurs de ces champs sont déterminées par le serveur.

| Statut | Interprétation / réaction client |
|---|---|
| 400 | Paramètres ou action invalides : corriger la saisie. |
| 401 / 403 | Session requise, origine refusée ou relation indisponible : vérifier le contexte. |
| 409 | Manche changée, salon plein ou solde insuffisant selon la route ; relire error et l’état courant. |
| 410 | Article saisonnier qui n’est plus en vente. |
| 503 | Service indisponible ou conflits épuisés : conserver le brouillon et réessayer. |

Les routes fonctionnelles renvoient Cache-Control: no-store. Les exemples ne demandent aucun accès manuel au cookie HttpOnly ; le navigateur le transmet dans son contexte habituel.

# Transitions, horloge et concurrence.

Les fonctions fresh, begin, advance, submit et view de lib/game.ts séparent le calcul de la partie de sa persistance HTTP.

## Machine d’états

| Transition | Déclencheur |
| --- | --- |
| empty vers countdown | Démarrage d’un salon. |
| countdown vers guess | Fin du compte à rebours. |
| guess vers reveal | Délai écoulé ou tous les joueurs ont trouvé. |
| reveal vers guess / podium | 5 secondes ; podium après la dixième manche. |
| podium vers countdown | Fin des 60 secondes de classement. |

advance retire les présences expirées et applique au plus la transition temporelle prévue pour l’instant courant. Une arrivée ou une synchronisation peut donc faire avancer une partie. Le client ne choisit jamais sa phase et ne décrète pas qu’une réponse est correcte.

## Écriture conditionnelle

```
UPDATE rooms
SET state = ?, revision = revision + 1
WHERE id = ? AND revision = ?;
```

La route relit l’état et sa révision, calcule l’action, puis tente cette mise à jour. Si aucune ligne n’a changé, elle recommence sur la nouvelle révision, dans une limite de 10 essais. Le lot comprend également les écritures de mesure autorisées et les récompenses. Un nonce telemetryCommit lie les crédits à la révision effectivement validée.

## Synchronisation côté navigateur

La partie est synchronisée toutes les 900 ms, sans lancer deux sync concurrentes. L’affichage du temps se rafraîchit toutes les 100 ms. lib/countdown.ts convertit deadline - serverNow en horloge monotone ; pour une même phase, un instantané tardif ne peut pas rallonger le chronomètre. Un numéro de session empêche une ancienne requête d’écraser le nouveau salon.

L’aperçu des salons est relu toutes les 3 secondes dans l’accueil et toutes les 10 secondes pendant une partie. La capacité et la présence sont revérifiées par le serveur lors de l’entrée, même si la carte affichée est ancienne.

# Crédits uniques et achats atomiques.

Il s’agit de monnaies de jeu, sans paiement réel intégré. Le serveur garde un livre d’écritures plutôt qu’un compteur de solde modifiable par le navigateur.

```
solde(player, currency) = SUM(delta)
possession = ecriture d’achat avec item et delta < 0

Identifiants d’ecriture :
round:{room}:{game}:{round}
solo:{room}:{game}:{player}:{seuil}
match:{room}:{game}:{player}
daily:{day}:{player}
purchase:{player}:{item}
```

## Récompenses de salon

coinProgress garde, pour chaque session, les manches tentées, les mots trouvés en solo et les premières places. Cette progression appartient au match et survit au départ/re-retour dans ce même match. roomCoinAwards calcule les nouveaux seuils et le bonus de podium. Le champ completed ne devient vrai que pour un joueur réellement présent à la fin et éligible.

## Atomicité et réessais

coinPrizeStatement utilise INSERT OR IGNORE avec un identifiant unique. Le SELECT d’insertion exige la bonne révision du salon et le bon telemetryCommit. Deux appels simultanés ou un réessai ne peuvent donc pas produire deux crédits pour la même récompense. La vue publique retire la progression complète et ne renvoie que le récapitulatif du demandeur.

## Achat et apparence

purchaseStatement vérifie le solde disponible et la limite saisonnière dans l’instruction SQL d’achat. Le prix, la monnaie et l’article viennent de COSMETICS. Le même identifiant purchase empêche un second débit pour un article déjà acquis. ownedAppearance retire les éléments payants non possédés lors d’une mise à jour du profil de jeu ou social.

Le remboursement historique Halloween utilise une grille figée et une date limite pour les achats en pièces. Une future hausse du catalogue ne doit pas réactiver cette remise pour de nouveaux acheteurs. Tests : tests/economy.test.mjs et tests/api.test.mjs.

# Journée de Paris et essais idempotents.

Les salons et le mot du jour partagent la normalisation des réponses, mais pas leur banque de questions, leurs points ou leurs récompenses.

## Identité du défi

dailyDate utilise Intl.DateTimeFormat avec Europe/Paris. La clé du défi est une date YYYY-MM-DD, indépendante du fuseau du téléphone. daily_challenges enregistre la question choisie pour cette date. daily_players conserve les essais et la révision de l’état du joueur, afin de reprendre la même tentative après actualisation.

## Réessayer une requête sans ajouter un essai

Le client associe un identifiant requestId à la proposition en attente. dailyGuess recherche cet identifiant dans les réponses déjà enregistrées ; si le même envoi a déjà été traité, il conserve l’état. Une erreur réseau ne doit donc pas inciter à générer immédiatement un nouvel identifiant pour le même envoi en cours.

## Classement et clôture

Le classement porte d’abord sur les joueurs qui ont résolu, puis sur le nombre d’essais et l’heure de résolution. Une participation réelle exige tries > 0. settleDailyStatement classe toute la journée fermée avant de créer les crédits. Le crédit daily:{day}:{player} est unique. La clôture est traitée lors des appels du service ; aucun ordonnanceur nocturne dédié n’est nécessaire pour la logique actuelle.

| Fonction | Rôle |
| --- | --- |
| dailyClues | Indices à partir de 3 et 6 erreurs. |
| dailyMasked | Révèle une voyelle à partir de 9 erreurs. |
| dailyGuess | Réponse normalisée et protection contre le même requestId. |
| settleDailyStatement | Répartition définitive des étoiles des journées antérieures. |

Le minuit de Paris ne correspond pas à une heure UTC fixe toute l’année. L’événement Halloween se termine le 10 novembre 2026 à 00 h à Paris, soit le 9 novembre à 23 h UTC. Conserver une date explicite avec son décalage lors des prochaines saisons.

# Assets, contours et superpositions.

Le personnage est un assemblage SVG d’images transparentes et d’éléments natifs. Le costume n’est pas une image opaque remplaçant tout le personnage.

## Références du pipeline

| Fichier / dossier | Utilité |
| --- | --- |
| app/components/Avatar.tsx | Composition, expression et ordre des couches. |
| lib/puppet-morphology.ts | Cibles adaptées aux différentes formes de tête. |
| lib/puppet-registration.* | Repères d’alignement des illustrations. |
| lib/costumes.ts + costume-assets.json | Placement des costumes et ouverture du visage. |
| accessory-assets.json / hair-assets.json | Index des accessoires et coiffures générés. |
| scripts/puppet-render.mjs | Rendu du vrai composant pour contrôles et illustrations. |

## Règles de composition

Le visage conserve sa silhouette native à l’intérieur de l’ouverture du costume. La couleur de peau ne remplit pas toute l’ouverture rectangulaire de l’asset. Les cheveux courts sous un couvre-chef utilisent un masque de tête ; les longueurs des tresses et carrés restent adaptées. Les broches et effets sont placés au-dessus du costume. Les lunettes et moustaches respectent la profondeur du nez.

## Ajouter ou corriger un asset

1. Créer une version séparée de l’image source, conserver son alpha et éviter de modifier les identifiants d’articles existants.
2. Préparer les WebP avec les scripts de packaging adaptés et mettre à jour l’index d’assets concerné.
3. Vérifier les contours sur toutes les formes de visage, puis tester les combinaisons cheveux, couvre-chef et costume.
4. Contrôler les états animés : yeux, bouche, premier à trouver et victoire. Réviser les coordonnées d’ancrage plutôt que d’étirer arbitrairement l’image.

Les tests d’avatars et les images rendues par script vérifient la géométrie. Ils ne prouvent pas que le rendu final d’une page mobile ou d’une modale est correct dans le navigateur.

# Identité, demandes et filtrage serveur.

La présence dans un salon, l’amitié acceptée et la visibilité de présence sont trois conditions distinctes. Un code ami n’est pas un moyen d’authentification.

## Demande depuis le classement

```
POST /api/friends
{
  "action": "request-room",
  "room": "cinema",
  "target": "1234abcd"
}
```

Le demandeur doit avoir son profil social et être présent dans ce salon. Le serveur résout le préfixe de huit caractères parmi les joueurs dont la présence date de moins de 12 secondes, en exigeant une correspondance unique. Il vérifie les blocages, prépare si nécessaire le contact public puis crée une demande pending. Un profil existant conserve ses choix de visibilité.

## Amitié et présence

Seul le destinataire peut passer une demande à accepted. Les demandes croisées sont conservées dans une direction unique. locate n’accepte que les amis autorisés avec un salon visible ; le nombre de joueurs est revérifié avant de proposer une entrée. Le panneau actualise ses données toutes les 15 secondes quand le document est visible. Le contrôle de blocage reste côté serveur.

## Discussion de salon

communicate refuse le chat pendant guess avant la résolution du demandeur. Les messages écrits pendant guess sont marqués privés aux gagnants. view masque ces messages aux joueurs qui n’ont pas trouvé jusqu’à la révélation ; le verrouillage ne repose pas seulement sur un bouton grisé. Le tableau conserve les 100 derniers messages du salon. Le panneau du bas sélectionne uniquement kind = chat ; les réponses restent dans le fil principal.

Les messages sont affichés comme texte React, sans HTML utilisateur injecté. L’effacement retire les messages identifiés par le joueur et son identifiant public court. La discussion n’est ni un canal chiffré de bout en bout ni une messagerie privée persistante.

# Construire, déployer et revenir en arrière.

Procédure de reprise technique. Les chemins des helpers dépendent de l’environnement Sites installé ; ne pas copier de jeton de publication dans un script versionné.

1. Restaurer les sources et le verrou de dépendances. Utiliser le helper d’installation Sites pour installer l’environnement prévu par le projet.
2. Vérifier .openai/hosting.json : conserver l’identifiant du Site et le binding D1 DB. Ne pas recréer un autre Site pour une simple mise à jour.
3. Déclarer les secrets d’administration dans l’environnement, sans les inscrire dans le dépôt. Compléter les paramètres publics de lib/publication.ts.
4. Si nécessaire, générer puis appliquer les migrations avec le circuit de déploiement prévu. Tester l’évolution sur une copie des données avant la production.
5. Exécuter TypeScript et les tests du module modifié. Construire avec le helper build-site.mjs. Ouvrir un aperçu quand l’infrastructure le permet.
6. Synchroniser la source, enregistrer une version à partir du même commit et de son archive, puis déployer cette version. Attendre un statut terminal succeeded.
7. Mettre à jour GitHub main et SOURCE_HISTORY.json ; conserver le lien entre la révision publiée et les sources.

## Retour en arrière

Identifier une version de code connue comme fonctionnelle et compatible avec le schéma D1 courant. Redéployer cette version par les outils d’hébergement. Vérifier un accès à un salon, un solde, une relation d’ami et le défi quotidien. Une migration incompatible impose une procédure de restauration de données préparée ; le retour de code seul n’annule pas les transactions.

## Surveillance minimale

Suivre les réponses 503, les échecs d’écriture, les conflits épuisés et les erreurs de configuration de l’administration. Comparer l’activité entre périodes uniquement sur un périmètre de consentement comparable. La journalisation ne doit pas exposer les cookies, codes secrets ou clés de statistiques privées.

Le service n’inclut pas une sauvegarde automatique D1 documentée ici. L’export, la rétention et la restauration de la base doivent être organisés avec l’hébergeur, puis réellement testés.

# Dépanner à partir d’un symptôme.

Collecter l’heure, le salon, la phase, la révision et le statut HTTP. Partager un exemple minimal sans cookie ni donnée privée. Une indisponibilité d’aperçu ne prouve pas un défaut du site publié.

| Symptôme | Vérification prioritaire |
|---|---|
| Salon bloqué / joueur disparu | Connectivité ; sync toutes les 900 ms ; présence 12 s ; fenêtre de reprise 10 s ; réponse expired. |
| Réponse rejetée après changement de manche | Comparer game et round envoyés avec le dernier instantané. Un 409 peut être une protection normale. |
| Pièces de fin non attribuées | 5 manches distinctes avec proposition non vide, présence au podium et écriture match du bon game. |
| Achat refusé | Cookie, monnaie de l’article, tarif serveur, solde du livre et date de fin saisonnière. |
| Ami introuvable ou bouton inactif | Demande acceptée, visibilité, blocage, présence commune et capacité du salon. |
| Discussion verrouillée | Phase guess et résolution du demandeur ; contrôler le filtrage serveur avant toute modification du bouton. |
| Administration indisponible | Variables de session et mot de passe robuste ; limitation des tentatives et statut 503 de configuration. |
| Image décalée / trou rectangulaire | Alpha, repères d’asset, forme de tête, masque et ordre des couches. |
| Données perdues après changement d’appareil | Identité liée au navigateur ; la clé statistique ne transfère pas collection ou amis. |

## Tests à lancer selon le module

Dans tests/ : game.test.mjs et api.test.mjs (jeu), economy.test.mjs (transactions), friends.test.mjs (social), room-discussion.test.mjs (panneau) et privacy.test.mjs (droits). Les rendus d’avatars contrôlent leur géométrie.

Un test SQLite vérifie une règle dans son environnement. La compilation vérifie la construction. Le statut de publication vérifie le déploiement annoncé. Le clavier, la lisibilité et la disposition PC/mobile exigent une recette visuelle.

Dimensionnement théorique : 30 clients synchronisés à 900 ms représentent environ 33 sync/s par salon, hors autres actions. Ce calcul n’est pas un test de charge ; mesurer le trafic et le coût D1 avant toute évolution.